Quick selection of commands for GTEK programmers
L - formatted list command, use start/end addresses for other than full eprom.
M - gives menu with a cr, or selection with proper letter (me = 2764)
OI- outputs intel hex file, use start/end addresses for other than full eprom.
OM- outputs motorola hex file. " "
OT- outputs Tektronic hex file. " "
P - program ascii hex characters, use start address if not at beginning.
R - read ascii hex characters, use start/end address if not full eprom.
T - toggle command:
    TB- toggle byte... toggle high/low byte when used with split eproms.
    TC- toggle compare mode.
    TI- toggle intelligent programming mode.
    TN- toggle checksum. may use start/end addresses.
    TR- toggle reset. resets all toggles.
    TS- toggle split mode. used to program 16 bit split eproms.
    TZ- toggle zap mode. used to erase some EE proms.
V - verify blank. use start/end addresses if necessary.
X - version command. gives version of programmer operating system.
$ - terminate any programmer operation.

examples of commands:
L ff lists contents of eprom in a formatted ascii hex dump from 0-ff hex.
l0-ff same results.
L0000-00ff same results.
OI3000-3FFF output an Intel Hex file from locations 3000 to 3FFF hex in
an eprom. You must realize that if you have a 2732 selected
for instance, you will actually get the contents of locations
0000 through 0fff hex. If you had a 2764 selected, you would
actually get the contents of 1000-1fff hex. If you had a 27128
selected you would actually get the contents of 3000-3fff hex.
The addresses are only significant to the programmer in relation
to the eprom size.
TN(cr) This causes the programmer to give you a 16 bit checksum
(addition of all 8 bit bytes without a 16 bit carry) of the
entire eprom selected. An empty 2764 gives a checksum of E000h.
TN ff gives the checksum of 0000 through 00ff hex in an eprom.
P1000 ffff$ programs the eprom selected at locations 1000h and 1001h with
the data ff and ff. Programmer returns to the command mode.

If you are not using our interface program, PGX, then the best way to use a modem program to capture data is to give the programmer a pending command
that the next character would be a (cr) for execution. Then give the modem

program the "capture incoming data" command to open your buffer. When you begin communication with the programmer again, send a (cr) to the programmer to begin the execution of the pending command. The (cr) will get put into the buffer that you opened, but if you are going to send it back to the programmer, it doesn't cause any problems. A (cr) causes the prompter to be reissued. You may also edit the file with a word processor like Wordstar in the Non-Document mode to remove the extraneous characters from the file. Make sure that you don't use any word processor that will change the file to any other kind of format. The programmer will promptly start telling you *SN err @ 0000 three times then appear to hang up. Any characters from the command mode that cause three syntax errors in a row will cause the programmer to go looking for a new baud rate. You must abort sending the file to the programmer and then send the programmer about 4 to 6 spaces to cause it to lock on to the baud rate again.

Wires necessary in RS-232 cable to communicate with GTEK programmers.

GTEK side (female) to DTE (IBM PC)

EG--1--------(optional but hooked in programmer)----1-EG (earth ground)
TXD-2---3-RXD
RXD-3---2-TXD
RTS-4---------(possibly optional)-------------------6-DSR (Data Set Ready)
CTS-5---20-DTR (Data Terminal Ready)
DSR-6---------(possibly optional)-------------------4-RTS (Ready To Send)
SG--7---7-SG (signal ground)
DTR-20--5-CTS (Clear To Send)
you may also jumper pin 6 DSR to pin 8 CD at both ends. This may be necessary if you use an IBM PC and other types of software.

SEARCHER.EXE

Searcher.EXE is a program designed to allow you to look up the Manufacturer's part number and see what menu selection to use on the Model 9800 or 9000. It is a very simple program and is not in the final form yet. There are so many new eproms and other devices that the 9800/9000 supports. Please check the list if you are not certain which menu selection and algorithm to use.

Example:

C:\GTEK\>searcher[enter]

Displays a list of manufacturers. Select the number of the manufacturer you desire (or the 9800) and a list of parts that can be programmed on the 9000/9800 will be displayed. The 9800 selection also lists the rotary dip selections. The highlighted selection is the suggested one. Use of another may not program or will damage the part. Follow on screen directions to quit or go to another.

You can add to the part list yourself by using a word processor in the non-document mode. I suggest you try using EDLIN, because if you do it right, you may not have to "fix" the file when you again enter searcher.

Bug Fixes?
1.01 First Release July, 1990
1.02 If Searcher "locks up" when you run it, but still puts the logon message on the screen, without saying "Mouse Driver present" or "NO mouse driver present" then try turning off all mouse driver checks by putting a /M or -M on the command line: C:\GTEK\>Searcher -m[enter]
Tandy 1000EX's are one that have this problem when there is no mouse attached. Dec, 1990

DEBUG.DOC
28 June 84, 1 April 86

This note will give some explaination on how to use Debug under MS/PC DOS with INTEL.HEX files. It will explain how to create an INTEL.HEX file with Ghex, our utility program, and how to change data in the file without creating a problem with the checksums. See the last section to modify Intel Hex files for even paragraph (16 byte) boundaries offset.

You can generate an INTEL.HEX file in several different ways. Under MS/PC DOS an assembler will usually create a .OBJ file and after it's linked you will have a .EXE or .COM file. A cross assembler, which can assemble source code for a processor other than the processor the assembler is running on can usually gene rate an INTEL.HEX file in addition to other types of files.

Examples to follow are for an eprom programmer like the 7128, 7228, 7956 or the 9000. You must have an INTEL.HEX file to start with, and we will make one if you don't have one. You must have DEBUG, PGMX, and GHEX on the same disk (to make explanation easier).

Using DEBUG, let's create a BINARY file to use GHEX on. In the following, the "A>" indicates the prompt your computer gave you. It may be "C>" on your computer, it doesn't make any difference, just make sure that the programs you need are on the same drive, or that you know how to "address" programs that are on a drive different from the one that you are logged onto. Also, a "<CR>" means that you are to strike the "return" key. A ";" means that a comment for your information follows and you are NOT to type that in too. Use the DOS manual to find the commands that are used for DEBUG to follow along with what's going on. In my book it's around chapter 12.

Type:

```
A>debug<CR>          ;enter debug.
-_                   ;You get a "-" prompt and a cursor flashing.
-f100,10ff,ff<CR>    ;fill memory from 0100 to 10ff (1000h bytes)
-ntstfl.img<CR>      ;fill the file control block with tstfl.IMG
-r cx<CR>            ;Tell cx register how many bytes to write
:1000<CR>            ;1000h (4096) bytes
-w<CR>               ;write 1kh bytes to file "tstfl.img"
-Q<CR>               ;Quit to the system (DOS)
A>
```

If you have a file on the disk already called tstfl.img, it will be overwritten. When we "GHEX" the file, GHEX will check to see if there is a file called tstfl.hex on the disk, and will not let you proceed until you rename tstfl.hex to something else or erase it. The following will take tstfl.img and create a new file called tstfl.hex on the disk with record addresses from 0000h to 0fffh, since we are not specifying any offset. If we want an offset we can specify it as in the second example.

Type:

```
(first example)
A>GHEX TSTFL.IMG      ;Creates TSTFL.HEX on the disk.
```

(second example)
A>GHEX TSTFL.IMG 100 ;Create file, records fm 0100h to 1100h

You can also get an INTEL.HEX file on the disk by reading one in from the programmer. Select a 2732 eprom with the menu command (for a 1kh file), or use the option line with PGMX. Don't put any eprom in the programming
socket.

Type:
A>PGMX TSTFL [R,MC]<CR> ;PGMX reads socket to INTEL.HEX file.
OR
<2732>^F
ENTER COMMAND -->TSTFL[R,MC<CR>

In either case the result will be a file by the filename TSTFL.HEX on the disk containing the contents of the eprom in the INTEL.HEX format with each record being an even 16d (10h) bytes long. We are now ready to operate.

The INTEL.HEX format is explained in your programmer manual. A quick recap of it follows eg. (spaces for clarity only!):

```
: 10 0000 00 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF 00<CR><lf>
1  2   3   4 5    < 16d (10h) data bytes >          6 checksum^  7  8
                                                 # ascii bytes
1- programmer recognizes colon as a command that
   an INTEL.HEX record is to follow.                          1
2- number of data bytes in this record 0 to 255d (00 to FFh)  2
3- load address of this record.                               4
4- Data type byte (nearly always 00).                         2
5- data bytes, in this case there are 16 BINARY data bytes   32
6- checksum                                                   2
7- carriage return (required for GTEK programmers)            1
8- line feed                    "                             1
                                        total ascii bytes =45
```

example file output by GHEX exactly 4096d (1000h) bytes long:

```
                :10000000FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF00
                :10001000FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF0

  (more records here... 20h-0FDFh...  shortened for brevity)

                :100FE000FFFFFFFFFFFFFFFFFFFFFFFFFFFFFF11
                :100FF000FFFFFFFFFFFFFFFFFFFFFFFFFFFFFF01
                :0000000000
```

This makes loading a hex file to memory very easy. DEBUG will accept an INTEL.HEX file directly and load it to the memory addresses specified in the file. BE EXTREMELY CAREFUL about overwriting active areas of your memory if you have any explicit segment addresses. You will probably have to load that file by hand or remove the segment records before you load that type of file directly to memory with DEBUG. Neither GHEX nor any of the programmers will create any segmented records for you to worry about.

Now we're ready to see an example or two about making changes in an Intel HEX file without upsetting any of the checksums. The trick is to change the (ascii) hex file to binary in memory with DEBUG and operating on it there. We can then save the modified memory (binary code) to the disk and regenerate the INTEL.HEX (ascii) type file with GHEX. To start, we are going to enter the DEBUG command state and do everything from there. See the exam ples below as we talk about it. Give DEBUG a filename with the "n" command. Now we want to "load" it to DEBUG's default free memory area. You should assume that the .HEX file will start at 0000h. This means that it will overwrite the file control area (seg:0000h-seg:00ffh). We will ignore the segment for the time being. It will change depending on how much memory you have available, if you have any interrupts running and a lot of other factors. What you should know about it is that DEBUG assumes that all memory from that segment:0000h up is free memory. You probably won't have to worry about it unless you have files for eproms larger than 27512's, or if you are using the INTEL EXTENDED HEX format files (16 bit). See the section on Debug in your PC-DOS manual.

We issue the command L1000 to load the file to the load address (+1000h) specified at the beginning of each hex record. We could have given an offset of 100h, but all you want to do is use some offset of at least 100h to keep from overwriting any of the file control blocks below that address.

Now, if DEBUG didn't give you any error messages, you can proceed to modify memory, otherwise you can look it up in the DOS manual under the chapter about error messages. There is one whole chap ter in the DOS manual about error messages issued by Debug, Edlin, etc. Refer to that chapter to interpet error messages.

You may do an "r" command to check to see what the size of your file is. The CX register will contain the number of bytes loaded to memory (if less than 65k) and BX will take the overflow (if over 65k). In our case the CX register will say 2000 and the BX register will have 0000. We will have to change the CX register to reflect that we really only have a 1kh file and then later, after we have modified our data, we will write from our offset address. Be sure to modify the CX register if it doesn't contain the proper number of bytes to write back to the disk.

```
Type:
A>DEBUG<CR>                          ;ENTER DEBUG
-NTSTFL.HEX<CR>                    ;ENTER FILENAME TO LOAD
-L1000<CR>                           ;LOAD FILE WITH OFFSET OF 1000H
-R<CR>                               ;LOOK AT CX IN PARTICULAR
<DISPLAY OF ALL THE REGISTERS HERE, LOOK AT CX IN PARTICULAR>
                    CX 2000        ;FILE SIZE HERE .... PLUS OFFSET
```

Now for an example, let's modify the addresses at 0AEFh IN THE EPROM. Now you might see why I chose to add an offset of 1000h. It makes finding the right address to modify easier. I know that the address I want to modify is 1000h + 0AEFh = 1AEFh. If I had chosen an offset of 100h, which is the first available address (and the address that the "W" command uses for a default), I would have to add 0100h + 0AEFh = 0BEFh. It's just easier to me

to use an offset where I don't really have to add anything together. Do what you are comfortable with or have "memory" for.

Note that <SP> means press the space bar like <CR> means press the carriage return key. When you use the "E" command, you must be extremely careful not to give DEBUG an address that you didn't mean to. Check your offset carefully. After you issue the Eaddr<CR> command, it will display the memory location and the contents. If you don't want to modify that location, simply press the space bar and you will advance to the next location without modification. If you do want to modify, enter the data and then press the space bar. You will see the next data dis played. When you are done, press the <CR> or <return> key after you press the space bar for the last time. When you are done with the modifications, you are ready to save the data to the disk. You can also look at the data with the "d" command. See example below.

Type (continuing from the previous page):

```
-E1AE0<SP>.FF<SP> .FF<SP> .FF<SP> .FF<SP><CR> ;we didn't modify
-E1AEF<SP>.FF 41<SP> .FF 42<SP> .FF 43<SP><CR>  ;we modified data
-D1AE0,AFF<CR>
SEGM:1AE0 FFFF FFFF FFFF FFFF FFFF FFFF FFFF FF41  ...............A
SEGM:1AF0 4243 FFFF FFFF FFFF FFFF FFFF FFFF FFFF  BC..............
-R CX<CR>           ;Change the CX register to reflect 1kh bytes
:1000<CR>
-NTSTFL.bin<CR>  ;give DEBUG a new file name to write to.
-W1000<CR>     ;write 1kh bytes from address 1000h to TSTFL.bin
WRITING 1000H BYTES
-Q<CR>              ;Quit DEBUG to go back to the DOS.
```

In the above example, after you check your data, change the CX register (if you haven't already) to reflect the actual number of binary hex bytes that you need to write to the disk. Give DEBUG the name of the file you want to write to. Usually you want to use a new name to be able to generate the new .HEX file with GHEX without having to rename or erase the old file. Use this method as a way of generating a "backup" of the old file. (Don't write over the old one!) Give the Write command now with the offset that you specified when you loaded the file originally. If you used an offset of 100h you don't have to tell DEBUG that. If it's 0ffh or 101h etc, you will have to be explicit. Please make a note that if you load a file like an EXE file, that the CS: register will be set to something other than what DEBUG is using. THIS IS DETERMINED ONLY BY THE EXTENSION OF THE FILENAME! If the extension is something other than .EXE or .HEX the CS: register will probably be the one that DEBUG is using. DEBUG writes data from where the CS: register is pointing, offset 0100H.

Now you should have a BINARY image of the original hex file, modified and on the disk with the new file name that you specified just before you wrote the file from DEBUG. We need to use INTEL.HEX (ascii) files with the programmer however, so use GHEX to generate the new .HEX file. If you need an offset from 0000h, be sure to specify it now. Remember that GHEX does not "Relocate" code for you. If you offset your code from some absolute address in the eprom, then it may not

work after you get through with it. If it's a data type eprom, like you would use with a pattern or mill machine, then you're probably safe. If it's program eprom data you just offset, then you're probably in trouble.

1- Convert to INTEL.HEX file: A>GHEX TESTA.DAT 1000<CR>

This converts our file to an INTEL.HEX file starting with a load address of 1000h. This is not significant to this 2732 eprom since it's highest address is 0FFFh. The data will "wrap-around" if you send it to the programmer. Address 0000h will be the same as 1000h in other words.

2- Load the file using DEBUG: A>DEBUG testa.HEX<CR>

We don't have to worry about overwriting the data in the addresses below 100h because we know that this file's addresses start at 1000h. If you had one that starts at 0000h you would use the "N" command and then "L100" it to memory with that offset. If you read in a file from the programmer then you will HAVE to use the offset method since the addresses will start at 0000h.

3- If we want to change a few bytes at address 1AB9h we then would issue the command: E1AB9<SP>. This will cause DEBUG to begin entering at that address. Type the data that you wish to be there. As an example let's enter the hex data that will display the word "HELLO" if you used Dump to display the hex and ascii data.

Type:
-E1ab9<SP> .FF 48<SP> .FF 45<SP> .FF 4B<SP> .FF 4B<SP> .FF 4F<SP><CR>
D1AB0,1ABF<CR>

1AB0 FF FF FF FF FF FF FF FF FF 48 45 4B 4B 4F FF FF .....HELLO..
-_

4- Now we need to save this changed information to the disk if we want to use it.

Type:
R CX<CR>
:1000<CR>
NNEWFILE.BIN<CR>
W1000<CR>

5- Use GHEX to convert back to a hex file we can send to the programmer.

Type:
A>GHEX NEWFILE.BIN<CR>

6- We should now have a file called NEWFILE.HEX on the disk, created by GHEX. We can send this file to the eprom programmer in this form.

A>PGMX NEWFILE [options]<CR>

The programmer will handle it from there. If you inserted and

selected the right eprom of course. It will probably let you know if you didn't. See the manual for instructions on using PGMX.

You may also get a binary file by using PGMX:

A>PGMX NEWFILE.IMG [%<CR>

Reads the selected eprom's data into the disk file newfile.img. The file size will exactly the size of the eprom, eg. 2732 = 4096 bytes, etc. If you also specify boundaries with the "@" command, you will get a file the size of what you specified, from the specified start address to the specified end address.

Some summary...

DEBUG commands:

Nfilespec.ext load file control block with your filespec.ext

L Read a file specified by the N command starting at 100h or in the case of an INTEL.HEX file, at the specified load addresses.

L100 Read a specified file into memory with a 100h offset. If it's an INTEL.HEX file starting at F00h, 100h will be added to all addresses and will go to memory starting at 1000h. (F00h+100h)

Eaaaa Enter data starting at address aaaah, up to the address where you enter a <CR> as data.

Fssss,eeee,dd Fill memory starting at address ssssh up to and including address eeeeh with the data ddh.

See the instruction manual for the model 9000 programmer or a file called READ.ME for instructions on using GHEX and PGMX.

You can offset data in an Intel Hex file by specifying a Segment record just before the data you wish to offset. The programmer adds this offset to the address counter inside the programmer, so the apparent addresses that are being sent to the programmer appear the same, but are actually offset by the USBA sent by the segment record. Refer to the section on the Intel Hex format in the manual for an example of the format.

Here are some segment records you could add to your code to offset your data within the eprom without otherwise modifying the file.

```
:02 0000 02 0000 FC  ;usba record with segment set to 0000H
  ^  ^    ^  ^   ^---;checksum
  |  |    |  |-------;usba 0000H - FFFFH segment
  |  |    |----------;record type, 02 for USBA
  |  |---------------;load address- always 0
  |------------------;always 2 for a Segment record
```

```
:020000020001FB     ;  10H offset
:020000020002FA     ;  20H
:020000020003F9     ;  30H
:020000020004F8     ;  40H
:020000020005F7     ;  50H
:020000020006F6     ;  60H
:020000020007F5     ;  70H
:020000020008F4     ;  80H
:020000020009F3     ;  90H
:02000002000AF2     ;  A0H
:02000002000BF1     ;  B0H
:02000002000CF0     ;  C0H
:02000002000DEF     ;  D0H
:02000002000EEE     ;  E0H
:02000002000FED     ;  F0H
:020000020010EC     ; 100H OFFSET
:020000020011EB     ; 110H
:020000020020DC     ; 200H
:020000020040BC     ; 400H
:0200000200807C     ; 800H
:020000020100FB     ;1000H
:020000020200FA     ;2000H
:020000020400F8     ;4000H
:020000020800F4     ;8000H

EXAMPLES
OFFSET A50H WITHIN AN EPROM:
:0200000200A557   <-----ADDED IN YOUR HEX FILE
:100000007F7F7F7F7F7F7F7F7F7F7F7F7F7F7F7F00

OFFSET 110H WITHIN AN EPROM (CODE ALREADY STARTS AT 10H):
:020000020010EC   <---- ADDED
:100010007F7F7F7F7F7F7F7F7F7F7F7F7F7F7FF0
:100020007F7F7F7F7F7F7F7F7F7F7F7F7F7F7FE0
:100030007F7F7F7F7F7F7F7F7F7F7F7F7F7F7FD0
(NEXT ADDRESS IS ACTUALLY 140H)

:020000020009F3 (OFFSET 90H, NEXT LOAD ADDRESS IS 90H+40H=D0H)
:100040007F7F7F7F7F7F7F7F7F7F7F7F7F7F7FC0
:100050007F7F7F7F7F7F7F7F7F7F7F7F7F7F7FB0
:100060007F7F7F7F7F7F7F7F7F7F7F7F7F7F7FA0
:100070007F7F7F7F7F7F7F7F7F7F7F7F7F7F7F90
:0000000000


------------- ***** ----------------------

Using COM3: and COM4:
If you want to use COM3: or COM4:, install PGMX7A for the appropriate
IRQ you want to use (COM1: uses IRQ4 as well as COM3: does, and COM2:
uses IRQ3 as well as COM4: does.) Install for COM1: (to use COM3:) and
COM2: to use as COM4: in other words, using PINSTAL7.

When this is done, use DEBUG to modify PGMX7A.COM as follows, Note that
the screen may not display exactly as shown here, but effect should be
the same:  (enter) means to hit the "enter" key.

C>debug pgmx7a.com(enter)
```

```
-d121,121(enter)
0121    F8      .
-e121 e8(enter)
-W(enter)
writing xxxx bytes
-Q(enter)
C>_
```

Summary of commands to get COM3 or COM4:
(location 121 should contain F8 when installed as COM1: or COM2:)

```
E121 E8
W
```

Now that was simple... wasn't it?